Exercise 8 – CI/CD and Testing

Eero Lainepää

1. Approach and Implementation

Overview

The goal of this exercise was to implement unit tests for a provided JavaScript library, set up a CI/CD pipeline using GitHub Actions, and report coverage using Coveralls.

Step-by-Step Process

I started by going through the library's source files alphabetically, excluding everything inside the .internal/ directory as instructed. I began by writing unit tests for the first few functions in the math category (add.js, ceil.js, clamp.js, divide.js) and verified they ran correctly locally:

```
npm run test
```

Once the first tests were working locally, I set up the GitHub Actions CI workflow early, before writing all the tests so that every subsequent push would automatically run the test suite. After the pipeline was running, I added coverage reporting using c8 and configured it to exclude the .internal/ directory.

Tests were organised into separate files by category:

- test/array.test.js – array functions
- test/math.test.js – math functions
- test/collection.test.js – collection functions
- test/object.test.js – object/type checking functions
- test/string.test.js – string functions
- test/util.test.js – utility functions

As I wrote tests I discovered bugs, which I fixed directly in the source files. All bugs are documented in Section 4.

Tools and Technologies

Tool	Purpose
Mocha	Test runner
Chai	Assertion library
c8	Code coverage
coverallsapp/github-action@v2	Coveralls integration
GitHub Actions	CI/CD pipeline

2. CI/CD Pipeline Configuration

The pipeline is configured in .github/workflows/ci.yml and triggers on every push to the repository.

```yaml
name: CI

on: [push]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '22'

      - name: Install dependencies
        run: npm install

      - name: Run tests with coverage
        run: npm run coverage
        continue-on-error: true

      - name: Send coverage to Coveralls
        uses: coverallsapp/github-action@v2
        with:
          github-token: ${{ secrets.GITHUB_TOKEN }}
          path-to-lcov: ./coverage/lcov.info
```

Key Decisions

- coverallsapp/github-action@v2 uses the built-in GITHUB_TOKEN, no manual token configuration needed
- c8 is configured in package.json to exclude .internal/ and generate both text and lcov reporters
- npx c8 mocha is used instead of c8 directly to ensure compatibility on both Windows and Linux

3. Files Tested and Skipped

Directly Tested Files

File	Coverage	Notes
add.js	100%	No issues
ceil.js	100%	No issues
clamp.js	100%	Bug found and fixed
compact.js	100%	Bug found and fixed
chunk.js	100%	Bug traced to slice.js
camelCase.js	100%	Bug found and fixed
castArray.js	100%	Bug found and fixed
countBy.js	100%	Bug found and fixed
defaultTo.js	100%	Bug found and fixed
difference.js	100%	No issues
divide.js	100%	Bug found and fixed
drop.js	100%	No issues
endsWith.js	100%	No issues
every.js	100%	No issues
filter.js	Covered	Bug found and fixed
get.js	Covered	No issues
isBoolean.js	Covered	No issues
isBuffer.js	Covered	Bug found and fixed
isDate.js	Covered	No issues
isEmpty.js	Covered	No issues
isObjectLike.js	Covered	No issues

File	Coverage	Notes
map.js	Covered	No issues
slice.js	Covered	Bug found and fixed

Files Not Directly Tested (Partial Coverage via Dependencies)

These files were reached through other tested functions but not tested directly:

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Lines
isSymbol.js	100	75	100	100	20
isTypedArray.js	100	50	100	100	9–27
keys.js	86.48	100	0	86.48	31–35
memoize.js	96.87	55.55	100	96.87	45–46
reduce.js	100	66.66	100	100	39
toFinite.js	100	70	100	100	30, 34–37
toNumber.js	89.23	61.53	100	89.23	49–50, 52–54, 56–57
toString.js	88.09	63.63	100	88.09	32–34, 36–37
words.js	97.36	85.71	100	97.36	35

Files Covered as Dependencies (Not Directly Tested)

File	Reason
isArrayLike.js	Used internally by other functions
isObject.js	Used internally by other functions
isLength.js	Used internally by other functions
isSymbol.js	Used internally by other functions
isTypedArray.js	Used internally by other functions
isObjectLike.js	Used internally by other functions
isArrayLikeObject.js	Used internally by other functions

4. Bugs Found

A total of 10 bugs were found during testing. All were fixed.

Bug 1 – slice.js: Incorrect end boundary handling
Description: When the end index exceeded the array length it was not clamped, causing chunk.js to produce incorrect results with extra undefined values in the last chunk.

Fix:

```
// Before
      if (end < 0) {
        end += length
      }
      length = start > end ? 0 : ((end - start) >>> 0)

// After
      if (end < 0) {
        end += length
      }
      end = end > length ? length : end
      length = start > end ? 0 : ((end - start) >>> 0)
```

Bug 2 – compact.js: Wrong initial index
Description: The result index started at -1 instead of 0, causing the first truthy value to be skipped.

Fix:

```
// Before
      let resIndex = -1;

// After
      let resIndex = 0;
```

Bug 3 – countBy.js: Counter initialised to 0 instead of 1
Description: When a new key was first encountered it was initialised to 0 instead of 1, meaning the first occurrence of every key was never counted.

Fix:

```
// Before
      baseAssignValue(result, key, 0);

// After
      baseAssignValue(result, key, 1);
```

Bug 4 – castArray.js: Missing empty argument handling
Description: Calling castArray() with no arguments threw an error instead of returning an empty array.

Fix:

```
// Before
      function castArray(...args) {
        const value = args[0]
        return Array.isArray(value) ? value : [value]
      }

// After
      function castArray(...args) {
        if (args.length === 0) return []  // Added
        const value = args[0]
        return Array.isArray(value) ? value : [value]
      }
```

Bug 5 – divide.js: Divides divisor by itself
Description: The divide operation divided the divisor by itself instead of the dividend by the divisor. divide(10, 2) returned 1 instead of 5.

Fix:

```
// Before
      (dividend, divisor) => divisor / divisor

// After
      (dividend, divisor) => dividend / divisor
```

Bug 6 – clamp.js: Bounds applied in wrong direction
Description: The clamp logic was inverted — numbers above the upper bound were kept as-is and numbers below the lower bound were not clamped correctly.

Fix:

```
// Before
      number = number >= upper ? number : upper
      number = number <= lower ? number : lower

// After
      number = number >= upper ? upper : number
      number = number <= lower ? lower : number
```

Bug 7 – camelCase.js: Leading space in result
Description: The reduce accumulator was initialised with ' ' (a space) instead of '', causing all camelCase results to have a leading space.

Fix:
```
// Before
      }, ' ');

// After
      }, '');
```

Bug 8 – defaultTo.js: NaN not treated as missing value
Description: The function only checked for null and undefined but not for NaN, meaning defaultTo(NaN, 42) returned NaN instead of 42.

Fix:
```
// Before
      return value == null ? defaultValue : value

// After
      return (value ?? NaN) !== value ? defaultValue : value
```

Bug 9 – filter.js: Result array initialised incorrectly
Description: The result array was initialised as [[]] (an array containing an empty array) instead of [], causing every filtered result to contain a stray empty array at index 0.

Fix:
```
// Before
      const result = [[]]

// After
      const result = []
```

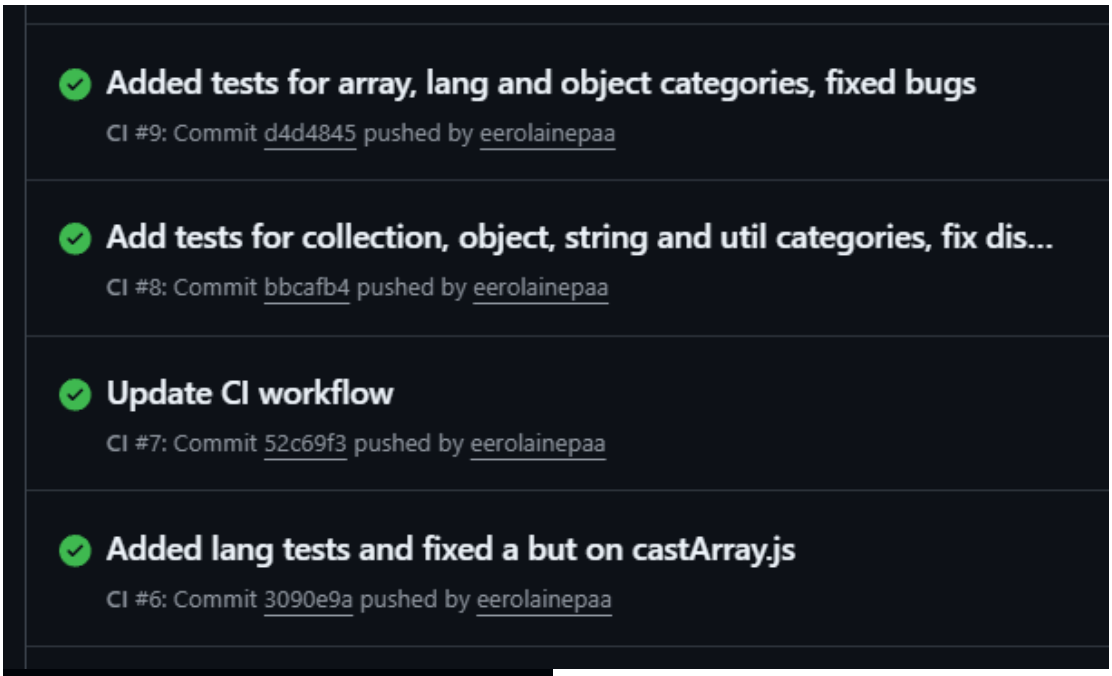
Bug 10 – isBuffer.js: CommonJS environment detection fails in ESM
Description: The freeExports/freeModule detection logic does not work in ES module environments, causing nativeIsBuffer to always be undefined and isBuffer to always return false.

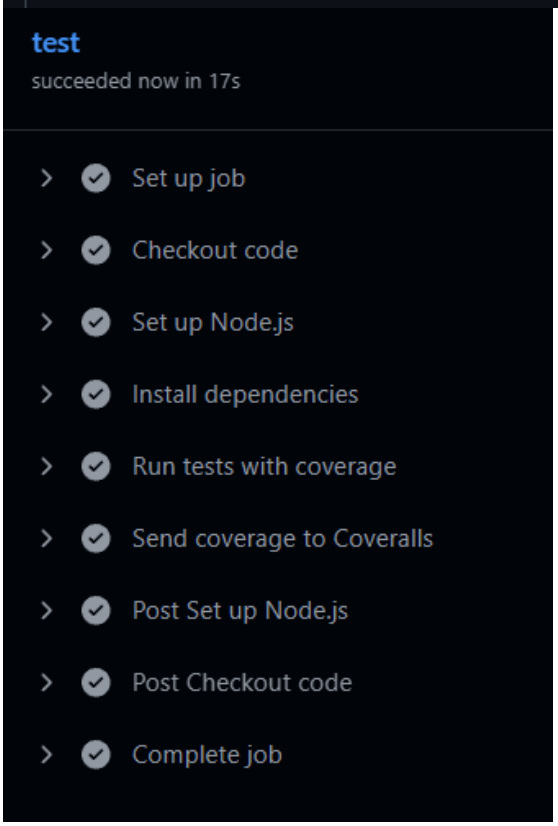
Fix:
```
// Before
      const Buffer = moduleExports ? root.Buffer : undefined

// After
      const Buffer = globalThis.Buffer
```

5. Test Results

GitHub Actions Workflow

All pipeline runs are visible in the Actions tab. Early runs failed due to the known bugs found during testing. After fixes were applied the pipeline runs green consistently.





Coverage Report

Final coverage results from Coveralls:

Metric	Result
Line coverage	91.08% (1992 / 2187 lines)
Branch coverage	86.94% (313 / 360 branches)

Coverage started at approximately 53% after the first test files were written and reached 91% after all categories were covered.

6. Final Verdict

The library is NOT ready for production.

While the test coverage achieved is high (91%), the library contained 10 bugs, and several of them were critical:

These are not edge-case issues, they affect the core functionality of fundamental functions. A library with this many logic errors in basic math and array operations is not suitable for production use without thorough review.

After applying the fixes documented in Section 4, the library behaves correctly for all tested cases and could be considered for production use, provided the remaining untested edge cases are also verified.